Elective Course

Course Code: CS4103

Autumn 2026-27

Lecture #07

Artificial Intelligence for Data Science

Module-2: PROBLEM SOLVING BY SEARCH

Introduction to Uninformed Search Techniques [Part-II]

Course Instructor:

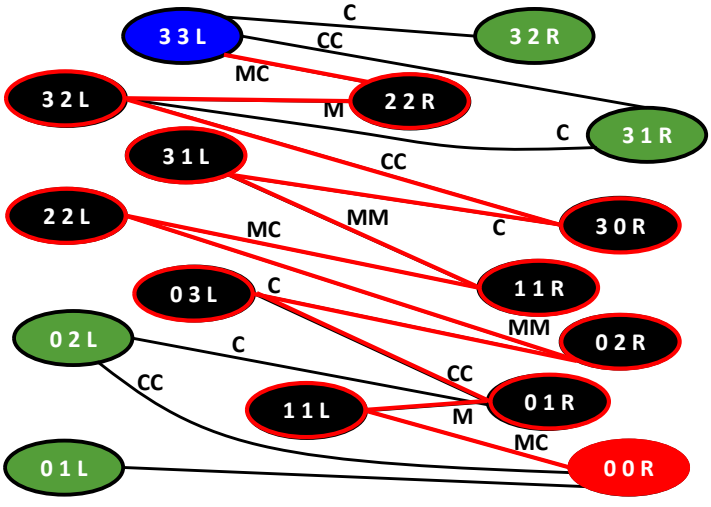
Dr. Monidipa Das

Assistant Professor

Department of Computational and Data Sciences

Indian Institute of Science Education and Research Kolkata, India 741246

Missionaries and Cannibals Problem Solution: DFS [scenario-1]



Soln: MC M CC C MM MC MM C CC M MC

L={(33L)}

L={(22R),(31R),(32R)}

L={(32L), (33L), (31R),(32R)}

L={(30R),(31R),(22R),(33L),(31R),(32R)}

L={(31L),(32L),(31R),(22R),(33L),(31R),(32R)}

L={(11R),(30R),(32L),(31R),(22R),(33L),(31R),(32R)}

L={(22L),(31L),(30R),(32L),(31R),(22R),(33L),(31R),(32R)}

L={(02R),(11R),(31L),(30R),(32L),(31R),(22R),(33L), (31R),(32R)}

L={(03L),(22L),(11R),(31L),(30R),(32L),(31R),(22R), (33L), (31R),(32R)}

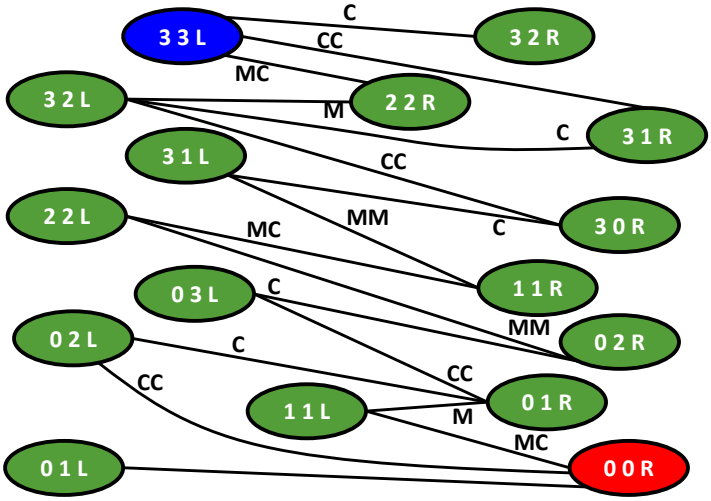
L={(01R),(02R),(22L),(11R),(31L),(30R),(32L),(31R), (22R), (33L), (31R),(32R)}

L={(11L)(02L),(03L),(02R),(22L),(11R),(31L),(30R), (32L),(31R),(22R), (33L), (31R),(32R)}

L={(00R),(01R),(02L),(03L),(02R),(22L),(11R),(31L), (30R),(32L),(31R),(22R), (33L), (31R),(32R)}

Goal State!

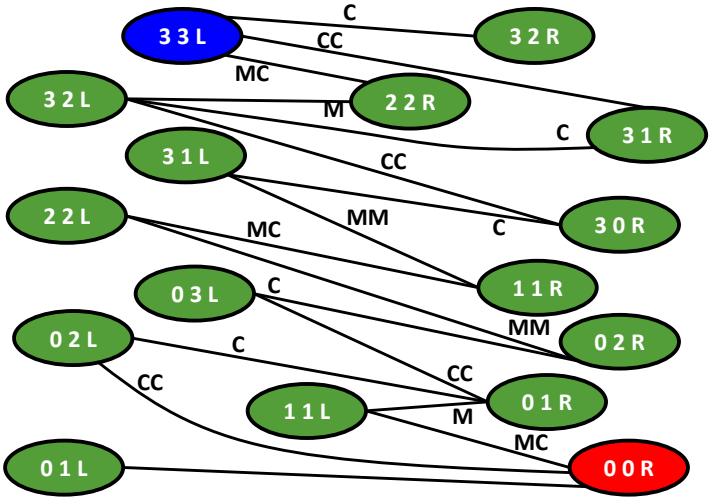
Missionaries and Cannibals Problem Solution: DFS
[scenario-2]



L={(~~33L~~)}
L={(~~32R~~),(31R),(22R)}
L={(~~33L~~),(31R),(22R)}
L={(~~32R~~),(31R),(22R),(31R),(22R)}
L={(~~33L~~),(31R),(22R),(31R),(22R)}
L={(~~32R~~),(31R),(22R),(31R),(22R),(31R),(22R)}
L={(~~33L~~),(31R),(22R),(31R),(22R),(31R),(22R)}
L={(~~32R~~),(31R),(22R),(31R),(22R),(31R),(22R),(31R),(22R)}
L={(~~33L~~),(31R),(22R),(31R),(22R),(31R),(22R),(31R),(22R)}
L={(~~32R~~),(31R),(22R),(31R),(22R),(31R),(22R),(31R),(22R),
(31R),(22R)}
L={(~~33L~~),(31R),(22R),(31R),(22R),(31R),(22R),(31R),(22R),
(31R),(22R)}
L={(~~32R~~),(31R),(22R),(31R),(22R),(31R),(22R),(31R),(22R),
(31R),(22R), (31R),(22R)}
. . . . . .
. . . . . .

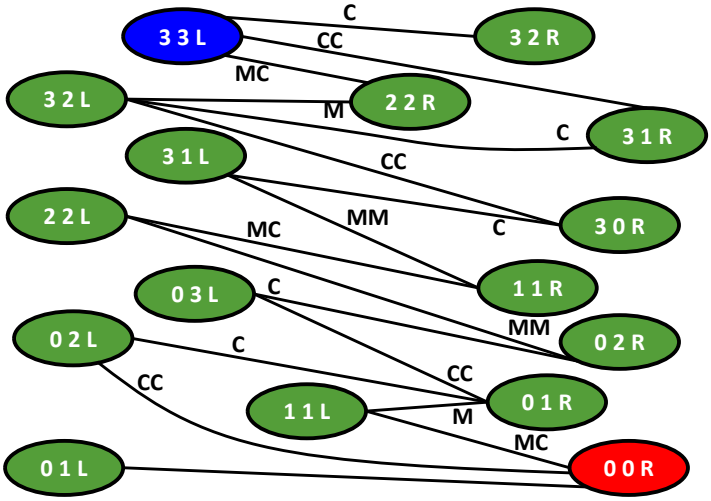
Infinite times…….. NO solution found

Missionaries and Cannibals Problem Solution: BFS



L={(~~33L~~)}
L={(~~22R~~),(31R),(32R)}
L={(~~31R~~),(32R), (32L), (33L)}
L={(~~32R~~), (32L), (33L),(32L),(33L)}
L={(~~32L~~), (33L),(32L),(33L),(33L)}
L={(~~33L~~),(32L),(33L),(33L),(30R),(22R),(31R)}
L={(~~32L~~),(33L),(33L),(30R),(22R),(31R),(22R), (31R),(32R)}
L={(~~33L~~),(33L),(30R),(22R),(31R),(22R),(31R),(32R),(30R),
(22R),(31R)}
L={(~~33L~~),(30R),(22R),(31R),(22R),(31R),(32R),(30R),(22R),
(31R),(22R),(31R),(32R)}
L={(~~30R~~),(22R),(31R),(22R),(31R),(32R),(30R),(22R),(31R),
(22R),(31R),(32R), (22R),(31R),(32R)}
L={(~~22R~~),(31R),(22R),(31R),(32R),(30R),(22R), (31R),
(22R),(31R),(32R),(22R),(31R),(32R),(31L),(32L)}
L={(~~31R~~),(22R),(31R),(32R),(30R),(22R),(31R),(22R),(31R),
(32R), (22R),(31R),(32R),(31L),(32L),(32L),(33L)}

Missionaries and Cannibals Problem Solution: BFS



L={(22R),(31R),(32R),(30R),(22R),(31R),(22R),(31R),(32R),(22R),(31R),(32R),(31L),(32L),(32L),(33L),(32L),(33L)}

L={(31R),(32R),(30R),(22R),(31R),(22R),(31R),(32R),(22R),(31R),(32R),(31L),(32L),(32L),(33L),(32L),(33L),(32L),(33L)}

L={(32R),(30R),(22R),(31R),(22R),(31R),(32R),(22R),(31R),(32R),(31L),(32L),(32L),(33L),(32L),(33L),(32L),(33L),(32L),(33L)}

L={(30R),(22R),(31R),(22R),(31R),(32R),(22R),(31R),(32R),(31L),(32L),(32L),(33L),(32L),(33L),(32L),(33L),(32L),(33L),(33L)}

L={(22R),(31R),(22R),(31R),(32R),(22R),(31R),(32R),(31L),(32L),(32L),(33L),(32L),(33L),(32L),(33L),(32L),(33L),(33L),(31L),(32L)}

. . . . . .

. . . . . .

. . . . . .

Huge computational time and space!

How to Avoid Exploring Redundant Path?

- The way to avoid exploring redundant paths is to **remember where one has been.**
- To do this, TREE-SEARCH algorithm is augmented with a data structure called the ***explored set*** (also known as the ***closed list***), which ***remembers every expanded node.***
- Newly generated nodes that match previously generated nodes—ones in the explored set or the frontier—can be discarded instead of being added to the frontier.
- The new algorithm, called **GRAPH-SEARCH**

Graph Search



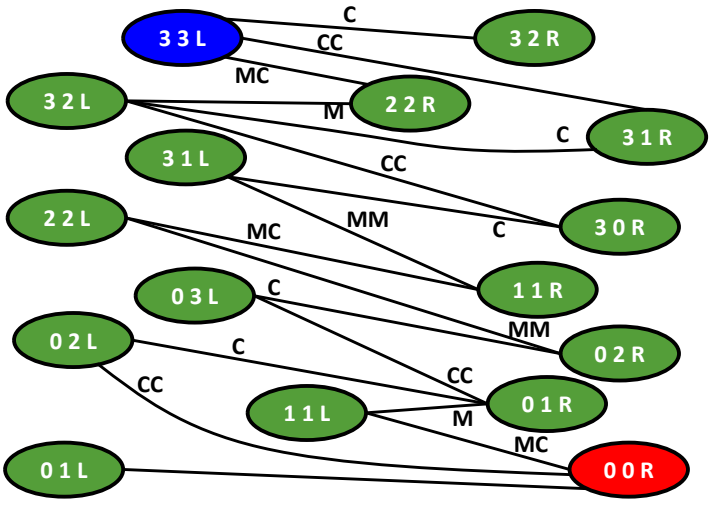
function GRAPH-SEARCH(*problem*) **returns** a solution, or failure
  initialize the frontier using the initial state of *problem*
  initialize the explored set to be empty
  loop do
    if the frontier is empty **then return** failure
    choose a leaf node and remove it from the frontier
    if the node contains a goal state **then return** the corresponding solution
    add the node to the explored set
    expand the chosen node, adding the resulting nodes to the frontier
      only if not in the frontier or explored set

✓GRAPH-SEARCH algorithm contains at most one copy of each state

✓the frontier separates the state-space graph into the explored region and the unexplored region, so that every path from the initial state to an unexplored state has to pass through a state in the frontier.

Missionaries and Cannibals Problem Solution: DFS [GRAPH-SEARCH]





L={(33L)}
E={}

L={(32R),(31R),(22R)}
E={(33L)}

L={(31R),(22R)}
E={(33L),(32R)}

L={(32L),(22R)}
E={(33L),(32R),(31R)}

L={(30R),(22R)}
E={(33L),(32R),(31R),(32L)}

L={(31L),(22R)}
E={(33L),(32R),(31R),(32L),(30R)}

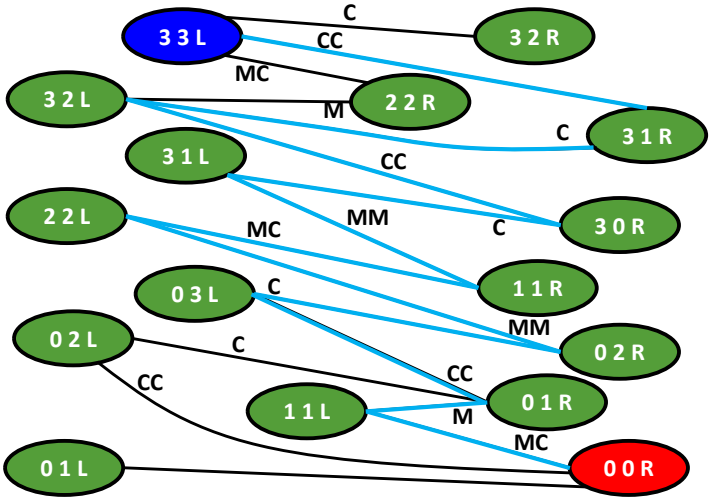
L={(11R),(22R)}
E={(33L),(32R),(31R),(32L),(30R),(31L)}

L={(22L),(22R)}
E={(33L),(32R),(31R),(32L),(30R),(31L),(11R)}

L={(02R),(22R)}
E={(33L),(32R),(31R),(32L),(30R),(31L),(11R),(22L)}

Missionaries and Cannibals Problem Solution: DFS [GRAPH-SEARCH]

9



L={(~~03L~~),(22R)}
E={(33L),(32R),(31R),(32L),(30R),(31L),(11R),(22L),(02R)}

L={(~~01R~~),(22R)}
E={(33L),(32R),(31R),(32L),(30R),(31L),(11R),(22L),(02R),(03L)}

L={(~~11L~~),(02L),(22R)}
E={(33L),(32R),(31R),(32L),(30R),(31L),(11R),(22L),(02R),(03L),(01R)}

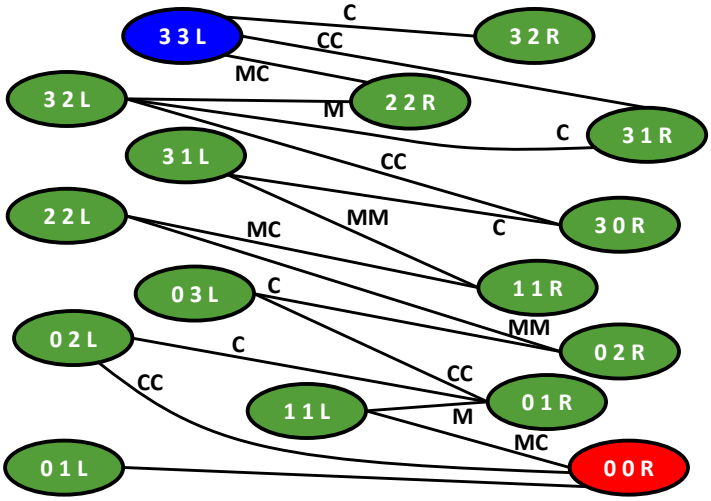
L={(~~00R~~),(02L),(22R)}
E={(33L),(32R),(31R),(32L),(30R),(31L),(11R),(22L),(02R),(03L),(01R),(11L)}

G (goal state) found!

Soln: CC C CC C MM MC MM C CC M MC

Missionaries and Cannibals Problem Solution: BFS [GRAPH-SEARCH]

10



L={(~~33L~~)}
E={}

L={(~~22R~~),(31R),(32R)}
E={(33L)}

L={(~~31R~~),(32R), (32L)}
E={(33L),(22R)}

L={(~~32R~~), (32L)}
E={(33L),(22R),(31R)}

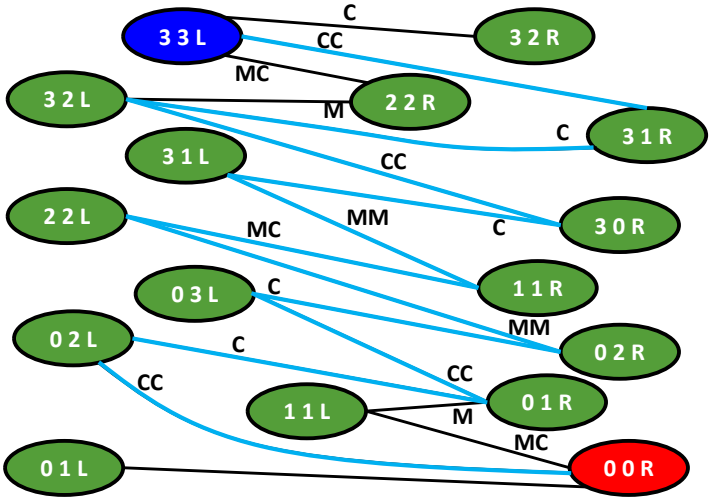
L={(~~32L~~)}
E={(33L),(22R),(31R),(32R)}

L={(~~30R~~)}
E={(33L),(22R),(31R),(32R),(32L)}

L={(~~31L~~)}
E={(33L),(22R),(31R),(32R),(32L),(30R)}

L={(~~11R~~)}
E={(33L),(22R),(31R),(32R),(32L),(30R),(31L)}

Missionaries and Cannibals Problem Solution: BFS [GRAPH-SEARCH]



Soln: CC C CC C MM MC MM C CC C CC

L={~~(22L)~~}
E={(33L),(22R),(31R),(32R),(32L),(30R),(31L),(11R)}

L={~~(02R)~~}
E={(33L),(22R),(31R),(32R),(32L),(30R),(31L),(11R),(22L)}

L={~~(03L)~~}
E={(33L),(22R),(31R),(32R),(32L),(30R),(31L),(11R),(22L),(02R)}

L={~~(01R)~~}
E={(33L),(22R),(31R),(32R),(32L),(30R),(31L),(11R),(22L),(02R),(03L)}

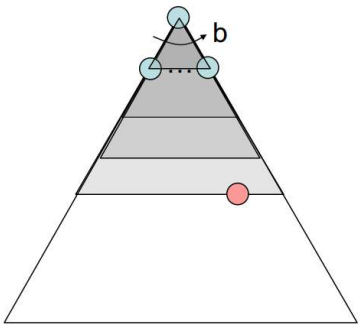
L={~~(02L)~~,(11L)}
E={(33L),(22R),(31R),(32R),(32L),(30R),(31L),(11R),(22L),(02R),(03L),(01R)}

L={~~(11L)~~,(00R)}
E={(33L),(22R),(31R),(32R),(32L),(30R),(31L),(11R),(22L),(02R),(03L),(01R),(02L)}

L={~~(00R)~~}
E={(33L),(22R),(31R),(32R),(32L),(30R),(31L),(11R),(22L),(02R),(03L),(01R),(02L),(11L)} **G (goal state) found!**

Iterative Deepening Depth-First Search (IDDFS)

- **Idea:** get DFS's space advantage with BFS's time / shallow-solution advantages
  - Run a DFS with depth limit 0. If no solution…
  - Run a DFS with depth limit 1. If no solution…
  - Run a DFS with depth limit 2. If no solution…
  - Run a DFS with depth limit 3. …..



- **Isn't that wastefully redundant?**
  - Generally most work happens in the deepest level searched, so not so bad

Iterative Deepening Depth-First Search



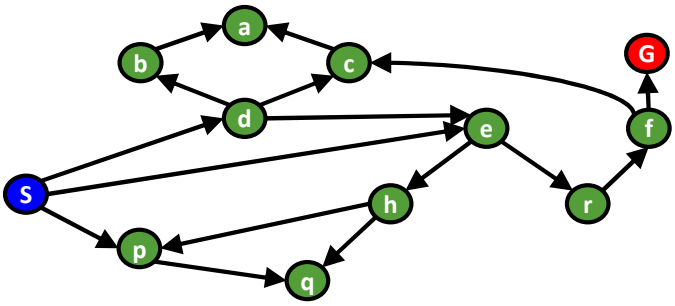

```
function ITERATIVE-DEEPENING-SEARCH(problem) returns a solution, or failure
    for depth = 0 to ∞ do
        result ← DEPTH-LIMITED-SEARCH(problem, depth)
        if result ≠ cutoff then return result
```

```
function DEPTH-LIMITED-SEARCH(problem, limit) returns a solution, or failure/cutoff
    return RECURSIVE-DLS(MAKE-NODE(problem.INITIAL-STATE), problem, limit)

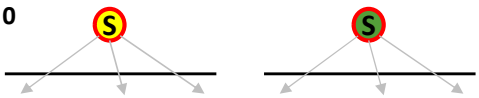
function RECURSIVE-DLS(node, problem, limit) returns a solution, or failure/cutoff
    if problem.GOAL-TEST(node.STATE) then return SOLUTION(node)
    else if limit = 0 then return cutoff
    else
        cutoff_occurred? ← false
        for each action in problem.ACTIONS(node.STATE) do
            child ← CHILD-NODE(problem, node, action)
            result ← RECURSIVE-DLS(child, problem, limit − 1)
            if result = cutoff then cutoff_occurred? ← true
            else if result ≠ failure then return result
        if cutoff_occurred? then return cutoff else return failure
```


Iterative Deepening Depth-First Search

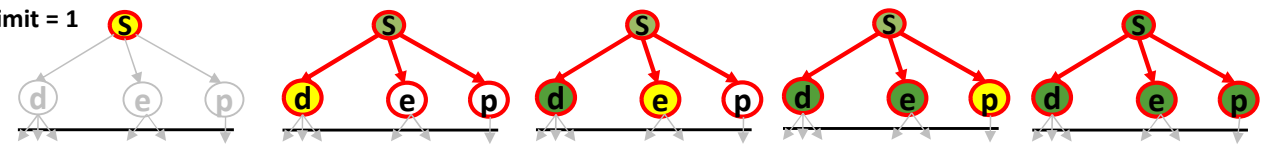




Limit = 0



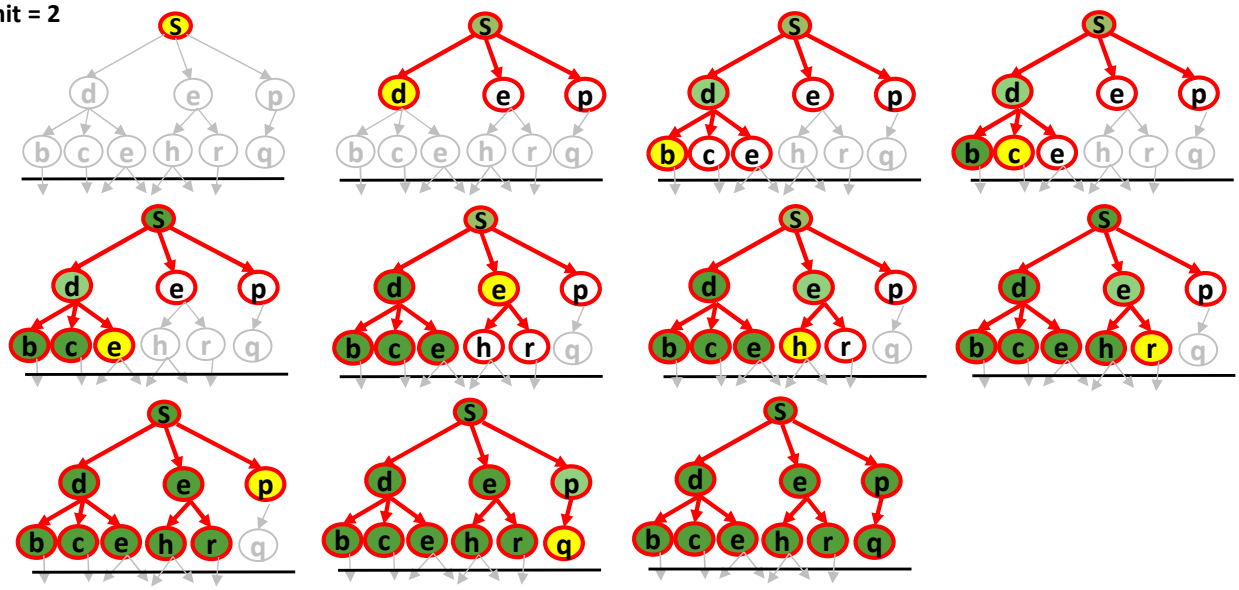
Limit = 1



Iterative Deepening Depth-First Search



Limit = 2



Iterative Deepening Depth-First Search



Complete? Yes

Time complexity? $db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$

Space complexity? $O(bd)$

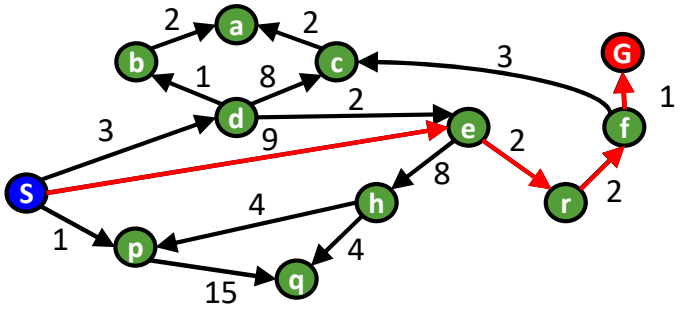
Optimal? Only if step costs are all identical

In general, iterative deepening is the preferred uninformed search method when the search space is large and the depth of the solution is not known

Optimality of Breadth-First Search

17

- **Is BFS optimal?**
  - It depends
  - Breadth-first search always finds the shallowest goal state
  - The path to that goal state, however, may have a higher cost than one to a deeper goal state



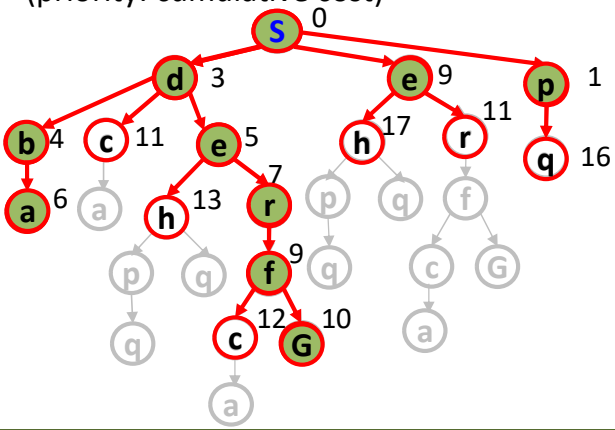
- **If we are looking for least-cost solutions, breadth-first is suboptimal** unless all step costs are identical

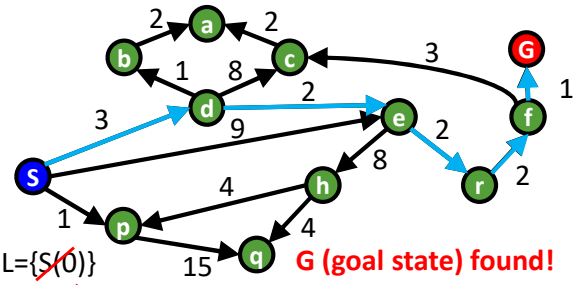
✓ BFS finds the shortest path in terms of number of actions.
✓ It does not find the least-cost path.

Uniform-Cost Search (UCS)

18

- **Strategy:** Expand least-cost unexpanded node or the cheapest node
- **Implementation:** Fringe = **priority queue** (priority: cumulative cost)



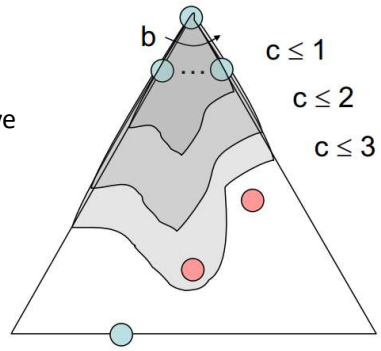


L={S(0)} **G (goal state) found!**
L={p(1), d(3), e(9)} **Path:**
L={d(3), e(9), q(16)} S→ d→ e→ r→ f→G
L={b(4), e(5), e(9), c(11), q(16)}
L={e(5), a(6), e(9), c(11), q(16)}
L={a(6), r(7), e(9), c(11), h(13), q(16)}
L={r(7), e(9), c(11), h(13), q(16)}
L={e(9), f(9), c(11), h(13), q(16)}
L={f(9), c(11), r(11), h(13), q(16), h(17)}
L={G(10), c(11), r(11), c(12), h(13), q(16), h(17)}

Uniform-Cost Search (UCS)

19

- **What nodes does UCS expand?**
  - Processes all nodes with cost less than cheapest solution!
  - If that solution costs C* and arcs cost at least ϵ, then the "effective depth" is roughly C*/ϵ
  - Takes time O($b^{C^*/\epsilon}$) (exponential in effective depth)
- **How much space does the fringe take?**
  - O($b^{C^*/\epsilon}$)
- **Is it complete?**
  - Assuming best solution has a finite cost and minimum arc cost is positive, yes!
- **Is it optimal?**
  - Yes!



When all step costs are equal, UCS is similar to BFS. However, BFS can stop as soon as it generates a goal, whereas UCS examines all the nodes at the goal's depth to see if one has a lower cost.

Comparing Uninformed Search Strategies

20

Criterion	Breadth-First	Uniform-Cost	Depth-First	Iterative Deepening
Complete?	Yes[a]	Yes[a,b]	No	Yes[a]
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^d)$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(bd)$
Optimal?	Yes[c]	Yes	No	Yes[c]

b, branching factor — d, depth of shallowest solution

m, depth of search tree — C^*, cost of optimal solution

[a] if b is finite [b] is step costs $> \epsilon$ for some $\epsilon > 0$ [c] is all step costs are the same

When to use what



- **Depth-First Search:**
  - Solutions are far from the initial state
- **Breadth-First Search**:
  - Some solutions are known to be shallow (not deep).
- **Iterative-Deepening Search**:
  - Computational space is limited and the shortest solution path is required
- **Uniform-Cost Search**:
  - Actions have varying costs
  - Least cost solution is required

  This is the only uninformed search that worries about costs.



Questions?